

Knowledge Representation

Lecture 6:

Part 1: Reasoning about Scenarios Part 2: Propositional Calculus

dr Anna Maria Radzikowska

Wydział Matematyki i Nauk Informacyjnych

Pokój 504, gmach MiNI

E-mail: A.Radzikowska@mini.pw.edu.pl

Basic assumptions

In contrast to previous approaches with *branching time* model, now we will consider action domains with *linear time*.

Basic assumptions

In contrast to previous approaches with *branching time* model, now we will consider action domains with *linear time*.

- Inertia law.
- Linear model of time (discrete time).
- Actions with duration; during performance of the action, values of fluents changed by the actions are unknown.
- Dynamic effects of actions – one action can invoke another one(s).
- Situation can trigger actions — some states may cause executing some actions.

Basic assumptions

In contrast to previous approaches with *branching time* model, now we will consider action domains with *linear time*.

- Inertia law.
- Linear model of time (discrete time).
- Actions with duration; during performance of the action, values of fluents changed by the actions are unknown.
- Dynamic effects of actions – one action can invoke another one(s).
- Situation can trigger actions — some states may cause executing some actions.

To represent these dynamic systems we will use action languages of the class *AL*. For simplicity we assume that each action is performed in 1 unit of time.

Action Language $\mathcal{AL}$ – syntax

As before, a *signature* is a pair $(\mathcal{F}, \mathcal{Ac})$, $\mathcal{F} \cap \mathcal{Ac} = \emptyset$.

Action Language $\mathcal{AL}$ – syntax

As before, a *signature* is a pair $(\mathcal{F}, \mathcal{Ac})$, $\mathcal{F} \cap \mathcal{Ac} = \emptyset$.

We have 5 types of statements:

- *fluent effect statement*:

$A \text{ causes } \alpha \text{ if } \pi$

Action Language $\mathcal{AL}$ – syntax

As before, a **signature** is a pair $(\mathcal{F}, \mathcal{A}_c)$, $\mathcal{F} \cap \mathcal{A}_c = \emptyset$.

We have 5 types of statements:

- **fluent effect statement:**

A causes α if π

- **action effect statement:**

A invokes B after d if π

where $A, B \in \mathcal{A}_c$, $\pi \in \text{Forms}(\mathcal{F})$, and $d \in \mathbb{N}$.

Intuitively, this statement says that the action B starts after d timepoints since the action A is completed, provided that the condition π holds when A starts.

Action Language $\mathcal{AL}$ (cont.)

• *release statement:*

A releases f if π

Action Language $\mathcal{AL}$ (cont.)

- *release statement:*

$A \text{ releases } f \text{ if } \pi$

- *trigger statement:*

$\pi \text{ triggers } A$

Intuitively, this statement says that the action A starts at any timepoint when the condition π holds.

Action Language $\mathcal{AL}$ (cont.)

- *release statement:*

$A \text{ releases } f \text{ if } \pi$

- *trigger statement:*

$\pi \text{ triggers } A$

Intuitively, this statement says that the action A starts at any timepoint when the condition π holds.

Action Language $\mathcal{AL}$ (cont.)

- *release statement:*

$A \text{ releases } f \text{ if } \pi$

- *trigger statement:*

$\pi \text{ triggers } A$

Intuitively, this statement says that the action A starts at any timepoint when the condition π holds.

A set of statements is called a *domain description*.

Action Language $\mathcal{AL}$ – scenarios

By an **action scenario** (**scenario**, for short) we mean a pair $Sc = (OBS, ACS)$, where

- OBS is the set of **observations**: $OBS = \{(\alpha_1, t_1), \dots, (\alpha_n, t_n)\}$, where $\alpha_i \in Forms(\mathcal{F})$ and $t_i \in \mathbb{N}$, $i = 1, \dots, n$
- ACS is the set of **action occurrences**: $ACS = \{(A_1, t_1), \dots, (A_n, t_k)\}$, where $A_i \in \mathcal{Ac}$ and $t_i \in \mathbb{N}$, $i = 1, \dots, k$.

Action Language $\mathcal{AL}$ – queries

A *query* in $\mathcal{AL}$ is an expression either of the form:

α at t when Sc A at t when Sc .

Intuitively, the 1st query states that the condition α holds at timepoint t when the scenario is carrying out, whereas the 2nd query says that the action A is executed at timepoint t when the scenario Sc is carrying out.

Action Language $\mathcal{AL}$ – semantics

Definition 5.1 A *structure* for $\mathcal{AL}$ is a system $S = (H, O, E)$ such that

- $H : \mathcal{F} \times \mathbb{N} \rightarrow \{0, 1\}$ is a *history function*

Action Language $\mathcal{AL}$ – semantics

Definition 5.1 A **structure** for $\mathcal{AL}$ is a system $\boxed{S = (H, O, E)}$ such that

- $H : \mathcal{F} \times \mathbb{N} \rightarrow \{0, 1\}$ is a **history function**
- $O : \mathcal{Ac} \times \mathbb{N} \rightarrow 2^{\mathcal{F}}$ is an **occlusion function**; for any action $A \in \mathcal{Ac}$ and for any timepoint $t \in \mathbb{N}$, $O(A, t)$ is the set of fluents under influence of the performance of A when executed from timepoint $t - 1$ to t

Action Language $\mathcal{AL}$ – semantics

Definition 5.1 A **structure** for $\mathcal{AL}$ is a system $\boxed{S = (H, O, E)}$ such that

- $H : \mathcal{F} \times \mathbb{N} \rightarrow \{0, 1\}$ is a **history function**
- $O : \mathcal{Ac} \times \mathbb{N} \rightarrow 2^{\mathcal{F}}$ is an **occlusion function**; for any action $A \in \mathcal{Ac}$ and for any timepoint $t \in \mathbb{N}$, $O(A, t)$ is the set of fluents under influence of the performance of A when executed from timepoint $t - 1$ to t
- $E \subseteq \mathcal{Ac} \times \mathbb{N}$ is an **actions occurrences relation**; if $(A, t) \in E$ then A occurs at timepoint t . We assume that for all $A, B \in \mathcal{Ac}$ and every $t \in \mathbb{N}$,

$$(A, t) \in E \ \& \ (B, t) \in E \implies A = B. \quad (1)$$

Action Language $\mathcal{AL}$ – semantics

Definition 5.1 A **structure** for $\mathcal{AL}$ is a system $\boxed{S = (H, O, E)}$ such that

- $H : \mathcal{F} \times \mathbb{N} \rightarrow \{0, 1\}$ is a **history function**
- $O : \mathcal{Ac} \times \mathbb{N} \rightarrow 2^{\mathcal{F}}$ is an **occlusion function**; for any action $A \in \mathcal{Ac}$ and for any timepoint $t \in \mathbb{N}$, $O(A, t)$ is the set of fluents under influence of the performance of A when executed from timepoint $t - 1$ to t
- $E \subseteq \mathcal{Ac} \times \mathbb{N}$ is an **actions occurrences relation**; if $(A, t) \in E$ then A occurs at timepoint t . We assume that for all $A, B \in \mathcal{Ac}$ and every $t \in \mathbb{N}$,

$$(A, t) \in E \ \& \ (B, t) \in E \implies A = B. \quad (1)$$

Note that the condition (1) guarantees that at most one action is executed at a time.

Semantics of $\mathcal{AL}$ (cont.)

For any structure $S = (H, O, E)$ for $\mathcal{AL}$, the history function H are extended for the set of all formulas according to rules well-known in propositional logic, i.e. for every timepoint $t \in \mathbb{N}$,

$$H^*(f, t) = H(f, t) \text{ for any } f \in \mathcal{F}$$

$$H^*(\neg\alpha, t) = 1 - H^*(\alpha, t)$$

$$H^*(\alpha \wedge \beta, t) = \min(H^*(\alpha, t), H^*(\beta, t))$$

$$H^*(\alpha \vee \beta, t) = \max(H^*(\alpha, t), H^*(\beta, t))$$

$$H^*(\alpha \rightarrow \beta, t) = \begin{cases} 0 & \text{iff } H^*(\alpha, t) = 1 \text{ \& } H^*(\beta, t) = 0 \\ 1 & \text{otherwise} \end{cases}$$

$$H^*(\alpha \equiv \beta, t) = \begin{cases} 1 & \text{iff } H^*(\alpha, t) = H^*(\beta, t) \\ 0 & \text{otherwise} \end{cases}$$

Semantics of $\mathcal{AL}$ (cont.)

Let $S = (H, O, E)$ be a structure for $\mathcal{AL}$, let $S_c = (OBS, ACS)$ be a scenario, and let D be a domain description. We say that S is a **structure for S_c wrt D** iff

- for each observation $(\alpha, t) \in OBS$, $H(\alpha, t) = 1$

Semantics of $\mathcal{AL}$ (cont.)

Let $S = (H, O, E)$ be a structure for $\mathcal{AL}$, let $S_c = (OBS, ACS)$ be a scenario, and let D be a domain description. We say that S is a **structure for S_c wrt D** iff

- for each observation $(\alpha, t) \in OBS$, $H(\alpha, t) = 1$
- $ACS \subseteq E$

Semantics of $\mathcal{AL}$ (cont.)

Denote: $fl(\alpha)$ – the set of fluents occurring in α .

- for each statement $(A \text{ *causes* } \alpha \text{ *if* } \pi) \in D$ and for each timepoint $t \in \mathbb{N}$,
if $H(\pi, t) = 1$ and $(A, t) \in E$, then $H(\alpha, t + 1) = 1$ and
 $fl(\alpha) \subseteq O(A, t + 1)$

Semantics of $\mathcal{AL}$ (cont.)

Denote: $fl(\alpha)$ – the set of fluents occurring in α .

- for each statement $(A \text{ *causes* } \alpha \text{ *if* } \pi) \in D$ and for each timepoint $t \in \mathbb{N}$, if $H(\pi, t) = 1$ and $(A, t) \in E$, then $H(\alpha, t + 1) = 1$ and $fl(\alpha) \subseteq O(A, t + 1)$
- for each statement $(A \text{ *releases* } f \text{ *if* } \pi) \in D$ and for each timepoint $t \in \mathbb{N}$, if $H(\pi, t) = 1$ and $(A, t) \in E$, then $f \in O(A, t + 1)$

Semantics of $\mathcal{AL}$ (cont.)

Denote: $fl(\alpha)$ – the set of fluents occurring in α .

- for each statement $(A \text{ *causes* } \alpha \text{ *if* } \pi) \in D$ and for each timepoint $t \in \mathbb{N}$, if $H(\pi, t) = 1$ and $(A, t) \in E$, then $H(\alpha, t + 1) = 1$ and $fl(\alpha) \subseteq O(A, t + 1)$
- for each statement $(A \text{ *releases* } f \text{ *if* } \pi) \in D$ and for each timepoint $t \in \mathbb{N}$, if $H(\pi, t) = 1$ and $(A, t) \in E$, then $f \in O(A, t + 1)$
- for each statement $(\pi \text{ *triggers* } A) \in D$ and for each timepoint $t \in \mathbb{N}$, if $H(\pi, t) = 1$, then $(A, t) \in E$

Semantics of $\mathcal{AL}$ (cont.)

Denote: $fl(\alpha)$ – the set of fluents occurring in α .

- for each statement $(A \text{ **causes** } \alpha \text{ **if** } \pi) \in D$ and for each timepoint $t \in \mathbb{N}$, if $H(\pi, t) = 1$ and $(A, t) \in E$, then $H(\alpha, t + 1) = 1$ and $fl(\alpha) \subseteq O(A, t + 1)$
- for each statement $(A \text{ **releases** } f \text{ **if** } \pi) \in D$ and for each timepoint $t \in \mathbb{N}$, if $H(\pi, t) = 1$ and $(A, t) \in E$, then $f \in O(A, t + 1)$
- for each statement $(\pi \text{ **triggers** } A) \in D$ and for each timepoint $t \in \mathbb{N}$, if $H(\pi, t) = 1$, then $(A, t) \in E$
- for each statement $(A \text{ **invokes** } B \text{ **after** } d \text{ **if** } \pi) \in D$ and for each timepoint $t \in \mathbb{N}$, if $H(\pi, t) = 1$ and $(A, t) \in A$, then $(B, t + d + 1) \in E$.

Semantics of $\mathcal{AL}$ (cont.)

Observe:

- Any change (in fluents' values) are allowed **only** in occlusion regions.

Semantics of $\mathcal{AL}$ (cont.)

Observe:

- Any change (in fluents' values) are allowed **only** in occlusion regions.
- Consequently, we will be interested in structures $S = (H, O, E)$ for $Sc = (OBS, ACS)$ wrt D , which occlusion functions O determine the smallest occlusion regions.

Semantics of $\mathcal{AL}$ (cont.)

Denote

Let $O_1, O_2 : X \rightarrow 2^Y$. We write

- $O_1 \preceq O_2$ iff $O_1(x) \subseteq O_2(x)$ for every $x \in X$.

Semantics of $\mathcal{AL}$ (cont.)

Denote

Let $O_1, O_2 : X \rightarrow 2^Y$. We write

- $O_1 \preceq O_2$ iff $O_1(x) \subseteq O_2(x)$ for every $x \in X$.
- $O_1 \prec O_2$ iff $O_1 \preceq O_2$ and $O_1 \neq O_2$.

Semantics of $\mathcal{AL}$ (cont.)

Denote

Let $O_1, O_2 : X \rightarrow 2^Y$. We write

- $O_1 \preceq O_2$ iff $O_1(x) \subseteq O_2(x)$ for every $x \in X$.
- $O_1 \prec O_2$ iff $O_1 \preceq O_2$ and $O_1 \neq O_2$.

Definition 5.2 Let $S = (H, O, E)$ be a structure for a scenario $S_C = (OBS, ACS)$ wrt a domain description D . We say that S is ***O-minimal*** iff there is no structure $S' = (H', O', E')$ for S_C wrt D such that $O' \prec O$.

Semantics of $\mathcal{AL}$ (cont.)

Definition 5.3 Let $S = (H, O, E)$ be a structure for a scenario $S_c = (OBS, ACS)$ wrt a domain description D . We say that S is a *model of S_c wrt D* iff

(M.1) S is O -minimal

Semantics of $\mathcal{AL}$ (cont.)

Definition 5.3 Let $S = (H, O, E)$ be a structure for a scenario $S_c = (OBS, ACS)$ wrt a domain description D . We say that S is a *model of S_c wrt D* iff

(M.1) S is O -minimal

(M.2) for every timepoint $t \in \mathbb{N}$,

$$\{f \in \mathcal{F} : H(f, t) \neq H(f, t + 1)\} \subseteq O(A, t + 1)\}$$

for some action $A \in \mathcal{A}_c$

Semantics of $\mathcal{AL}$ (cont.)

Definition 5.3 Let $S = (H, O, E)$ be a structure for a scenario $S_c = (OBS, ACS)$ wrt a domain description D . We say that S is a *model of S_c wrt D* iff

(M.1) S is O -minimal

(M.2) for every timepoint $t \in \mathbb{N}$,

$$\{f \in \mathcal{F} : H(f, t) \neq H(f, t + 1)\} \subseteq O(A, t + 1)\}$$

for some action $A \in \mathcal{A}_c$

(M.3) there is no structure $S' = (H', O', E')$ for S_c wrt D satisfying

(M.1)–(M.2) such that $E' \subset E$.

Semantics of $\mathcal{AL}$ (cont.)

Definition 5.3 Let $S = (H, O, E)$ be a structure for a scenario $S_c = (OBS, ACS)$ wrt a domain description D . We say that S is a **model of S_c wrt D** iff

(M.1) S is O –minimal

(M.2) for every timepoint $t \in \mathbb{N}$,

$$\{f \in \mathcal{F} : H(f, t) \neq H(f, t + 1)\} \subseteq O(A, t + 1)\}$$

for some action $A \in \mathcal{A}_c$

(M.3) there is no structure $S' = (H', O', E')$ for S_c wrt D satisfying

(M.1)–(M.2) such that $E' \subset E$.

We say that a scenario S_c is **consistent** wrt a domain description D iff there exists a model S of S_c wrt D ; otherwise it is called **inconsistent**.

Example 1: Inconsistency

Let D be a domain description with two actions, A and B , and let a scenario $Sc = (OBS, ACS)$ be given as

- $OBS = \emptyset$
- $ACS = \{(A, 1), (B, 1)\}$.

Example 1: Inconsistency

Let D be a domain description with two actions, A and B , and let a scenario $Sc = (OBS, ACS)$ be given as

- $OBS = \emptyset$
- $ACS = \{(A, 1), (B, 1)\}$.

Since A and B are to be executed parallel, Sc is inconsistent wrt any D (with the actions A and B) – there is no structure for Sc wrt any D .

Example 2: (In)consistency

Consider the following domain description D and two scenarios:

$Sc_1 = (OBS_1, ACS_1)$ and $Sc_2 = (OBS_2, ACS_2)$, given by

A **causes** f ;

B **causes** $\neg f$;

C **causes** g ;

A **invokes** C **after** 1.

- $OBS_1 = OBS_2 = \emptyset$

- $ACS_1 = \{(A, 1), (B, 3)\}$

- $ACS_2 = \{(A, 1), (B, 2)\}$.

Example 2: (In)consistency

Consider the following domain description D and two scenarios:

$Sc_1 = (OBS_1, ACS_1)$ and $Sc_2 = (OBS_2, ACS_2)$, given by

A **causes** f ;

B **causes** $\neg f$;

C **causes** g ;

A **invokes** C **after** 1.

• $OBS_1 = OBS_2 = \emptyset$

• $ACS_1 = \{(A, 1), (B, 3)\}$

• $ACS_2 = \{(A, 1), (B, 2)\}$.

For any $S = (H, O, E)$ for Sc_1 wrt D , we have $(A, 1), (B, 3) \in E$. Since $(C, 3) \in E$, Sc_1 is inconsistent wrt D .

Example 2: (In)consistency

Consider the following domain description D and two scenarios:

$Sc_1 = (OBS_1, ACS_1)$ and $Sc_2 = (OBS_2, ACS_2)$, given by

A **causes** f ;

B **causes** $\neg f$;

C **causes** g ;

A **invokes** C **after** 1.

• $OBS_1 = OBS_2 = \emptyset$

• $ACS_1 = \{(A, 1), (B, 3)\}$

• $ACS_2 = \{(A, 1), (B, 2)\}$.

For any $S = (H, O, E)$ for Sc_1 wrt D , we have $(A, 1), (B, 3) \in E$. Since $(C, 3) \in E$, Sc_1 is inconsistent wrt D .

For any $S = (H, O, E)$ for Sc_2 wrt D , $(A, 1), (B, 2), (C, 3) \in E$, so no inconsistency occurs.

Example 3: Modification of YSP

Consider the following domain description D :

LOAD **causes** *loaded*;
SHOOT **causes** $\neg$ *loaded*;
SHOOT **causes** $\neg$ *alive* **if** *loaded* $\wedge$ $\neg$ *hidden*;
LOAD **invokes** ESCAPE;
ESCAPE **releases** *hidden*.

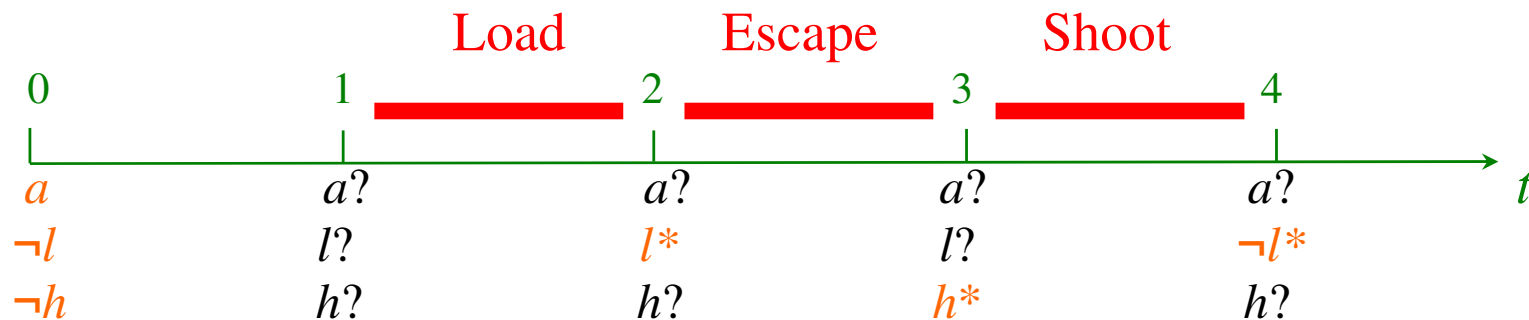
and a scenario $Sc = (OBS, ACS)$, where

$OBS = \{(alive \wedge \neg loaded \wedge \neg hidden, 0)\}$
 $ACS = \{(LOAD, 1), (SHOOT, 3)\}$.

Structures for S_C wrt D

There are two main classes of structures for S_C wrt D . Namely,

Class 1:



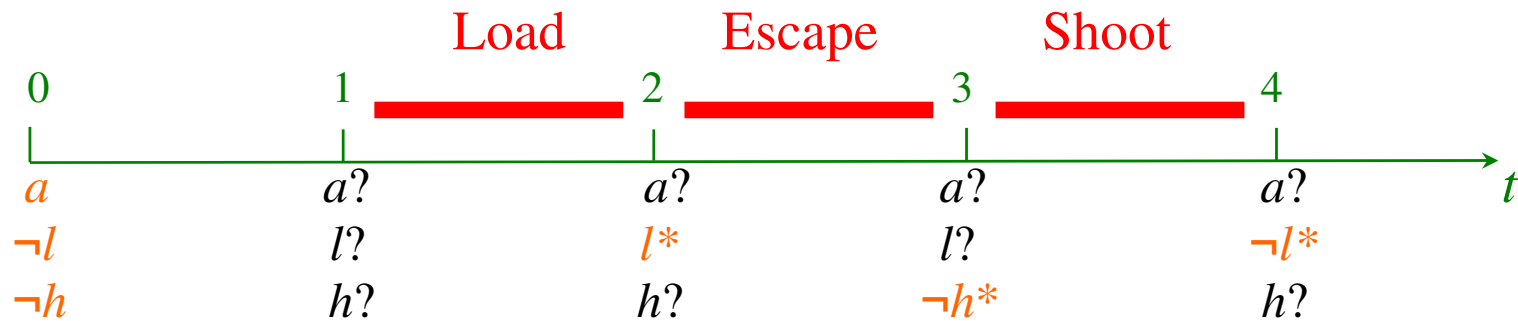
$$\{l\} \subseteq Occlude(\text{LOAD}, 2)$$

- Occlusion regions: $\{h\} \subseteq Occlude(\text{ESCAPE}, 3)$
 $\{l\} \subseteq Occlude(\text{SHOOT}, 4).$

- Occurrences of actions: $\{(\text{LOAD}, 1), (\text{ESCAPE}, 2), (\text{SHOOT}, 3)\} \subseteq E.$

Structures for S_c wrt D (cont.)

Class 2:

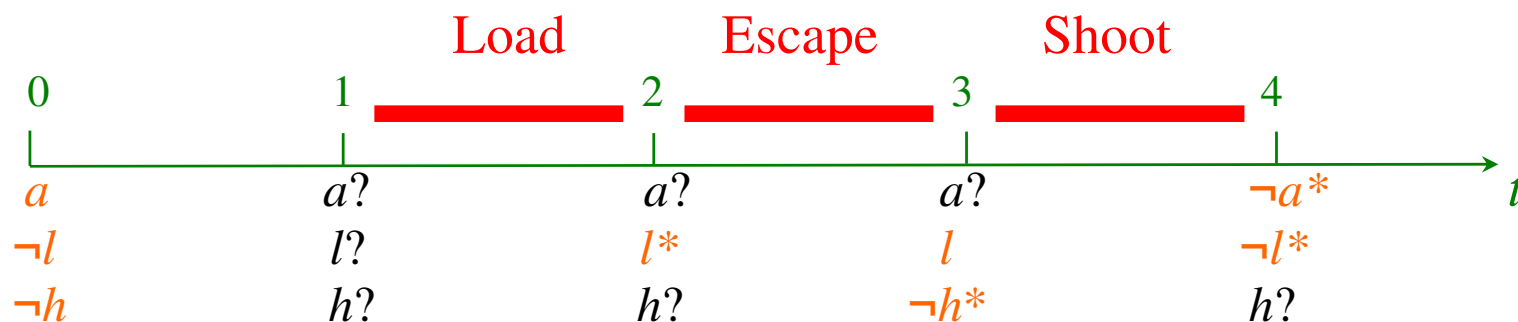


- $\{l\} \subseteq Occlude(\text{LOAD}, 2)$
- Occlusion regions: $\{h\} \subseteq Occlude(\text{ESCAPE}, 3)$
 $\{l\} \subseteq Occlude(\text{SHOOT}, 4).$
- Occurrences of actions: $\{(\text{LOAD}, 1), (\text{ESCAPE}, 2), (\text{SHOOT}, 3)\} \subseteq E.$

Structures for S_C wrt D (cont.)

Class 2 has two subclasses:

Subclass 1:



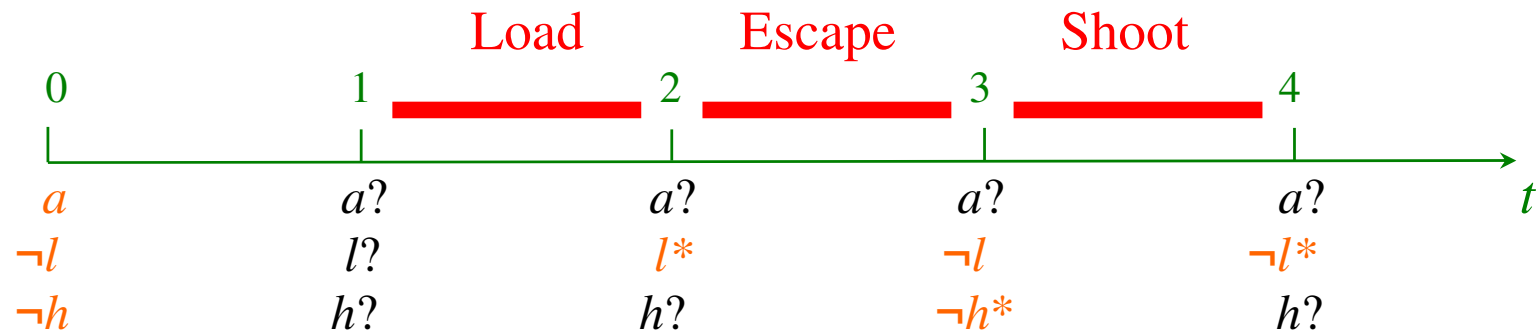
$$\{l\} \subseteq Occlude(\text{LOAD}, 2)$$

- Occlusion regions: $\{h\} \subseteq Occlude(\text{ESCAPE}, 3)$
 $\{a, l\} \subseteq Occlude(\text{SHOOT}, 4).$

- Occurrences of actions: $\{(\text{LOAD}, 1), (\text{ESCAPE}, 2), (\text{SHOOT}, 3)\} \subseteq E.$

Structures for S_c wrt D (cont.)

Subclass 2:

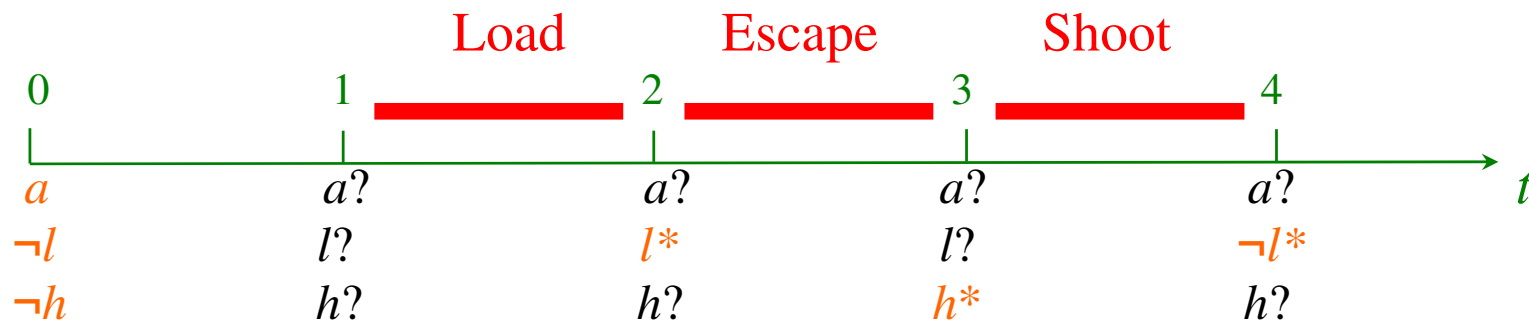


- $\{l\} \subseteq Occlude(\text{LOAD}, 2)$
- Occlusion regions: $\{h\} \subseteq Occlude(\text{ESCAPE}, 3)$
 $\{l\} \subseteq Occlude(\text{SHOOT}, 4).$
- Occurrences of actions: $\{(\text{LOAD}, 1), (\text{ESCAPE}, 2), (\text{SHOOT}, 3)\} \subseteq E.$

Structures for S_C wrt D (cont.)

Two main classes of O -minimal structures for S_C wrt D :

Class 1:



$$Occlude(\text{LOAD}, 2) = \{l\}$$

- Occlusion regions: $Occlude(\text{ESCAPE}, 3) = \{h\}$

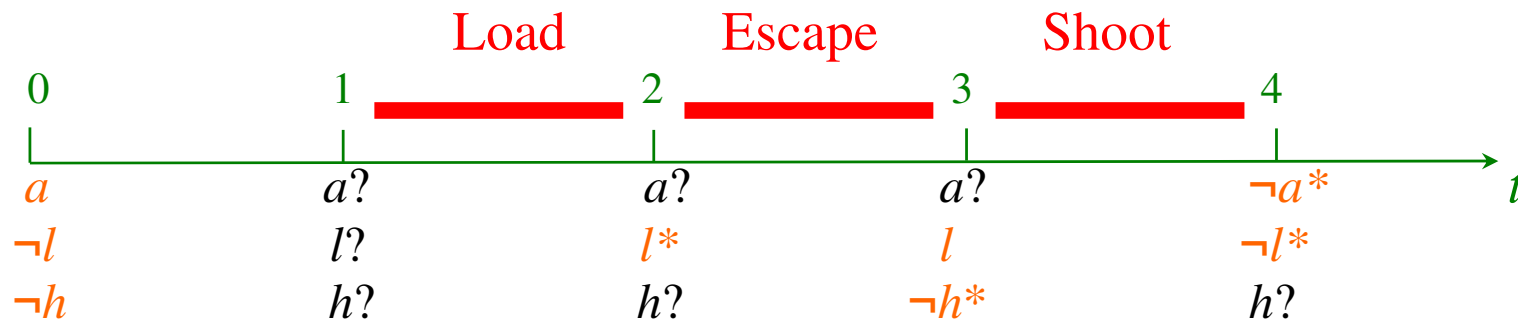
$$Occlude(\text{SHOOT}, 4) = \{l\}.$$

- Occurrences of actions: $\{(\text{LOAD}, 1), (\text{ESCAPE}, 2), (\text{SHOOT}, 3)\} \subseteq E$

Structures for S_C wrt D (cont.)

And two subclasses of the 2nd class:

1st subclass:



$$Occlude(\text{LOAD}, 2) = \{l\}$$

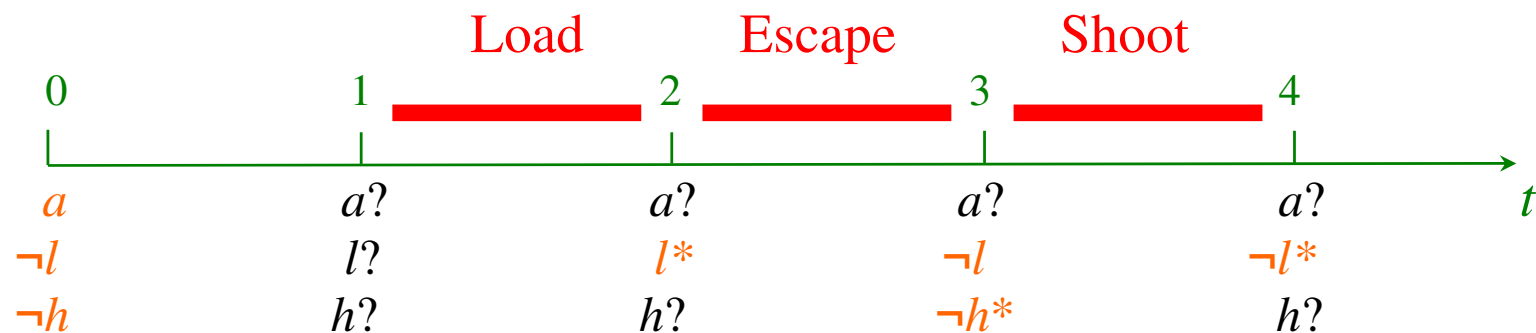
● Occlusion regions: $Occlude(\text{ESCAPE}, 3) = \{h\}$

$$Occlude(\text{SHOOT}, 4) = \{a, l\}.$$

● Occurrences of actions: $\{(\text{LOAD}, 1), (\text{ESCAPE}, 2), (\text{SHOOT}, 3)\} \subseteq E$.

Structures for S_c wrt D (cont.)

And the 2nd subclass of the class 1:



$$Occlude(\text{LOAD}, 2) = \{l\}$$

● Occlusion regions: $Occlude(\text{ESCAPE}, 3) = \{h\}$

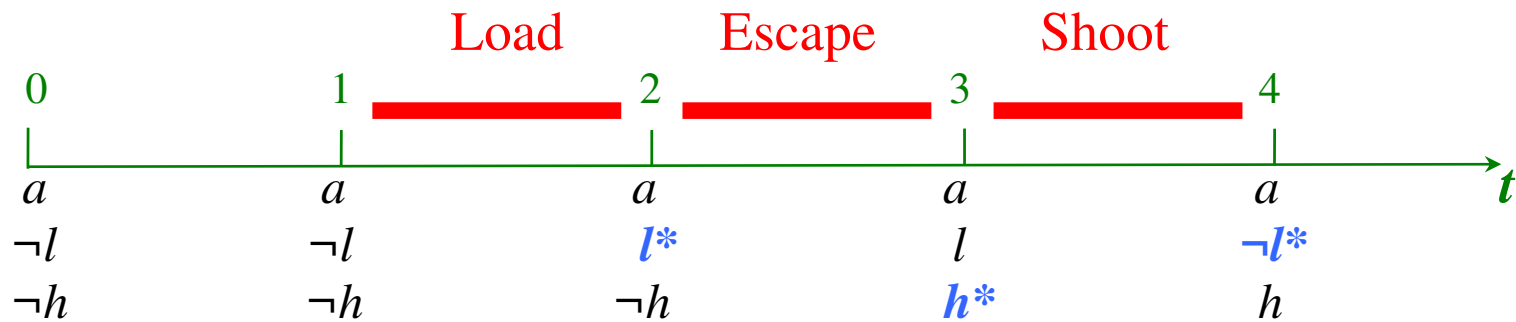
$$Occlude(\text{SHOOT}, 4) = \{l\}.$$

● Occurrences of actions: $\{(\text{LOAD}, 1), (\text{ESCAPE}, 2), (\text{SHOOT}, 3)\} \subseteq E.$

Models of S_C wrt D

There are two models of S_C wrt D :

1st model S_1 :



$$Occlude(\text{LOAD}, 2) = \{l\}$$

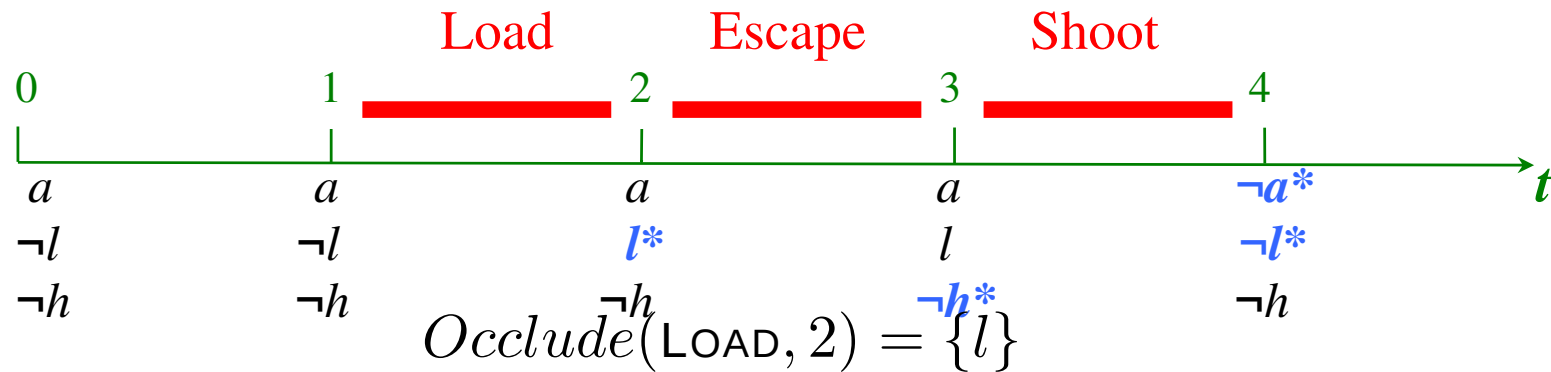
● Occlusion regions: $Occlude(\text{ESCAPE}, 3) = \{h\}$

$$Occlude(\text{SHOOT}, 4) = \{l\}.$$

● Occurrences of actions: $\{(\text{LOAD}, 1), (\text{ESCAPE}, 2), (\text{SHOOT}, 3)\} = E$.

Models of S_C wrt D

2nd model S_2 :



- Occlusion regions: $Occlude(\text{ESCAPE}, 3) = \{h\}$
 $Occlude(\text{SHOOT}, 4) = \{a, l\}.$
- Occurrences of actions: $\{(\text{LOAD}, 1), (\text{ESCAPE}, 2), (\text{SHOOT}, 3)\} = E.$

Queries of $\mathcal{AL}$

Let S_c be a scenario and let D be a domain description. We say that a query Q is a **consequence of S_c wrt D** , in symbols $D, S_c \models Q$, iff

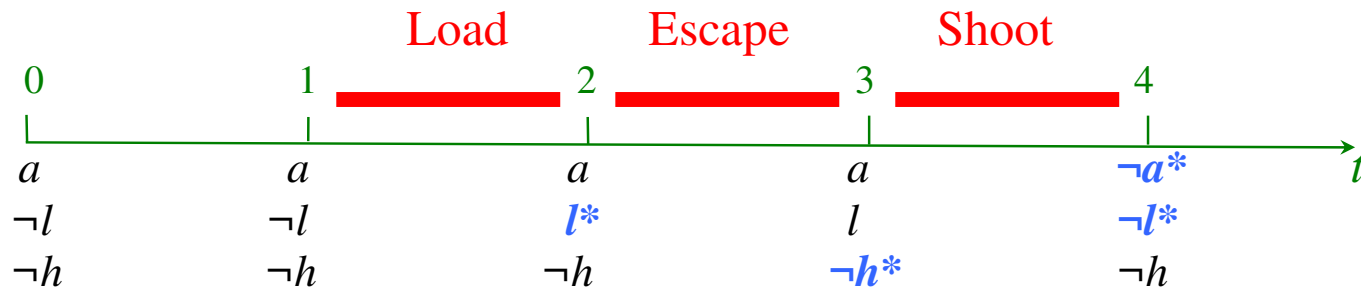
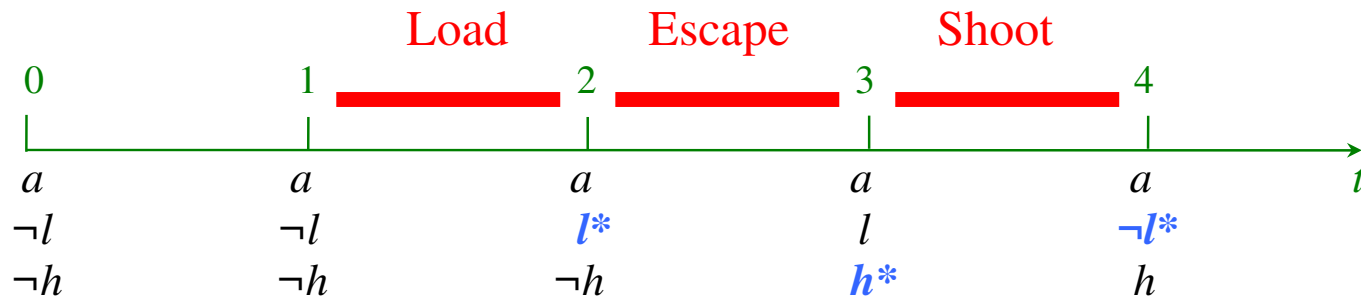
- if Q is of the form α **at** t **when** S_c , then for every model $S = (H, O, E)$ of S_c wrt D , it holds $H(\alpha, t) = 1$

Queries of $\mathcal{AL}$

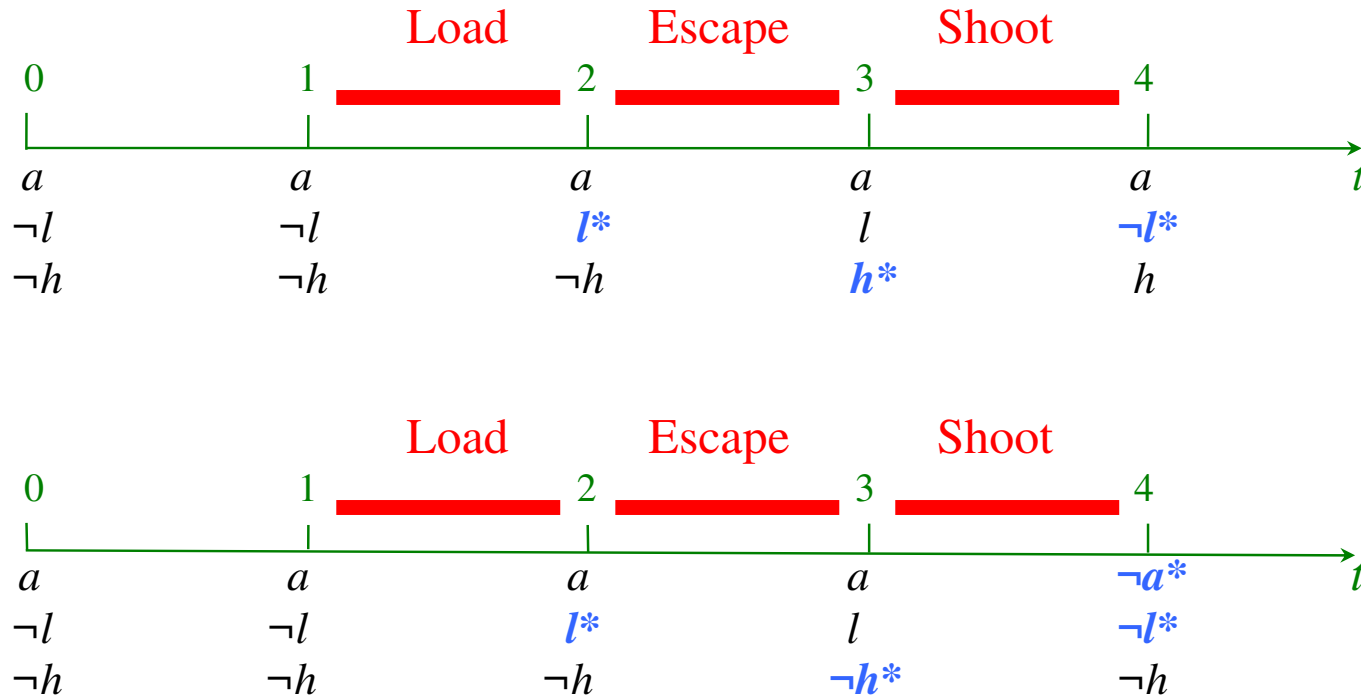
Let Sc be a scenario and let D be a domain description. We say that a query Q is a **consequence of Sc wrt D** , in symbols $D, Sc \models Q$, iff

- if Q is of the form α **at** t **when** Sc , then for every model $S = (H, O, E)$ of Sc wrt D , it holds $H(\alpha, t) = 1$
- if Q is of the form A **at** t **when** Sc , then for every model $S = (H, O, E)$ of Sc wrt D , it holds $(A, t) \in E$.

Example 3 (cont.)

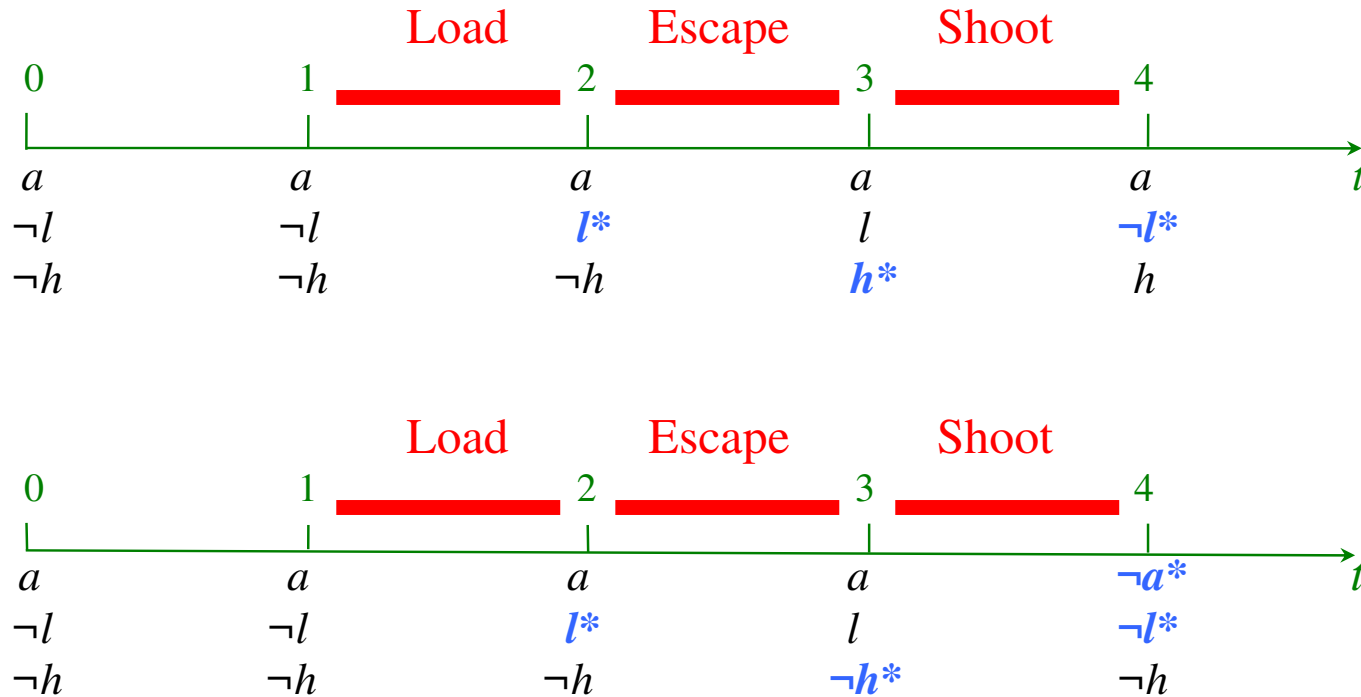


Example 3 (cont.)



$Sc, D \models \neg loaded$ **at** t **when** Sc for $t \geq 4$

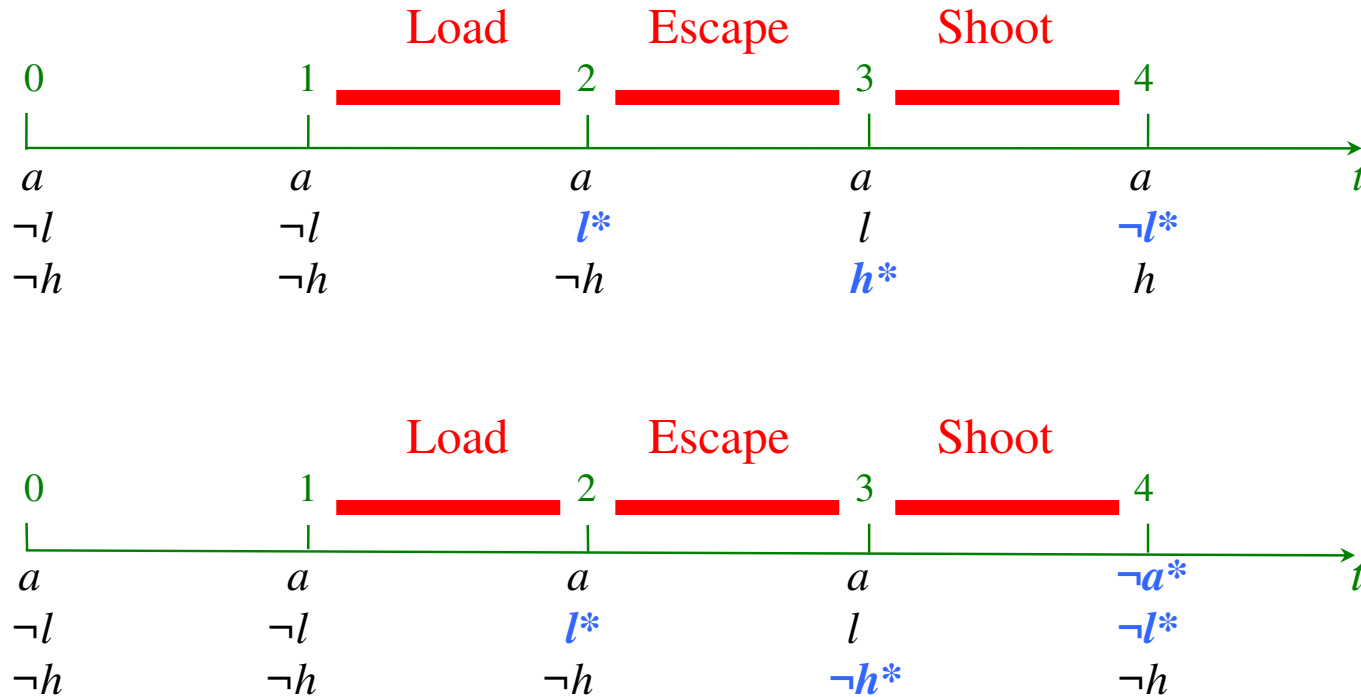
Example 3 (cont.)



$Sc, D \models \neg loaded$ **at** t **when** Sc for $t \geq 4$

$Sc, D \models \text{ESCAPE}$ **at** 2 **when** Sc

Example 3 (cont.)

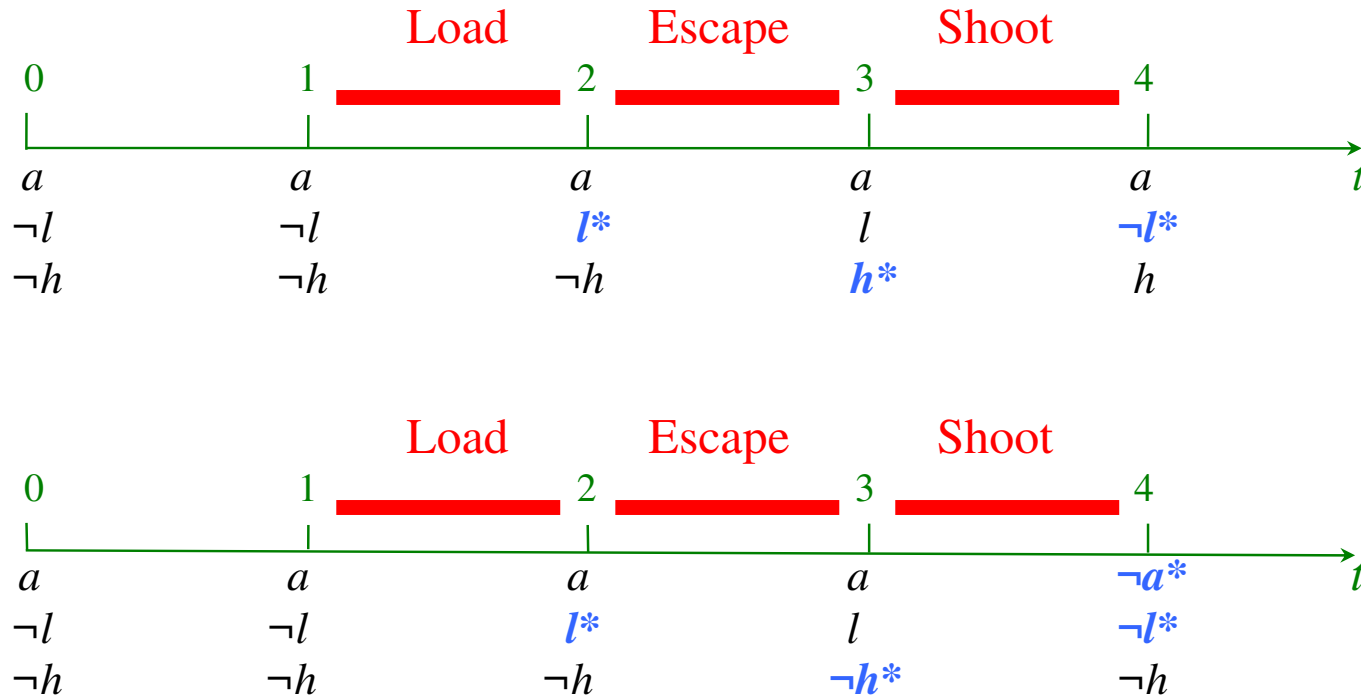


$Sc, D \models \neg loaded$ **at** t **when** Sc for $t \geq 4$

$Sc, D \models \text{ESCAPE}$ **at** 2 **when** Sc

$Sc, D \not\models alive$ **at** 4 **when** Sc

Example 3 (cont.)



$Sc, D \models \neg loaded$ **at** t **when** Sc for $t \geq 4$

$Sc, D \models ESCAPE$ **at** 2 **when** Sc

$Sc, D \not\models alive$ **at** 4 **when** Sc

$Sc, D \not\models \neg alive$ **at** 4 **when** Sc .

Foundations to Classical Logic

The notion of logic

By a *logic* we mean a triple $Log = (\mathcal{L}, \Sigma, \models)$, where

- $\mathcal{L}$ is a *language* of Log (i.e. the set of all formulas in Log)

The notion of logic

By a *logic* we mean a triple $Log = (\mathcal{L}, \Sigma, \models)$, where

- $\mathcal{L}$ is a *language* of Log (i.e. the set of all formulas in Log)
- Σ is the class of all frames used for interpretation of formulas

The notion of logic

By a **logic** we mean a triple $Log = (\mathcal{L}, \Sigma, \models)$, where

- $\mathcal{L}$ is a **language** of Log (i.e. the set of all formulas in Log)
- Σ is the class of all frames used for interpretation of formulas
- $\models: 2^\Sigma \rightarrow 2^\mathcal{L}$ is the consequence mapping which for each set $\mathfrak{M}$ of frames determines the set of formulas **satisfied** in every frame from $\mathfrak{M}$.

The notion of logic

By a **logic** we mean a triple $Log = (\mathcal{L}, \Sigma, \models)$, where

- $\mathcal{L}$ is a **language** of Log (i.e. the set of all formulas in Log)
- Σ is the class of all frames used for interpretation of formulas
- $\models: 2^\Sigma \rightarrow 2^\mathcal{L}$ is the consequence mapping which for each set $\mathfrak{M}$ of frames determines the set of formulas **satisfied** in every frame from $\mathfrak{M}$.

Classical logic:

- Propositional logic
- First order propositional calculus.

Propositional Calculus

A language of a propositional logic (PC) is determined by the following disjoint sets of symbols:

- a set Var of propositional variables
- the truth constant $\top$
- logical connectives $\neg$ and $\rightarrow$
- parentheses (and).

Propositional Calculus

A language of a propositional logic (PC) is determined by the following disjoint sets of symbols:

- a set Var of propositional variables
- the truth constant $\top$
- logical connectives $\neg$ and $\rightarrow$
- parentheses (and).

The set of all propositional formulas $\mathcal{L}$ (*language*) is the smallest set of the following expressions:

- $Var \subseteq \mathcal{L}$
- $\top \in \mathcal{L}$
- if α, β are formulas, then so are $\neg\alpha$ and $\alpha \rightarrow \beta$.

Propositional Calculus (cont.)

The remaining symbols are defines as:

- the truth constant $\perp : \perp \stackrel{def}{=} \neg \top$

Propositional Calculus (cont.)

The remaining symbols are defines as:

- the truth constant $\perp$: $\perp \stackrel{def}{=} \neg \top$
- logical connectives:
 - disjunction : $\alpha \vee \beta \stackrel{def}{=} \neg \alpha \rightarrow \beta$
 - conjunction : $\alpha \wedge \beta \stackrel{def}{=} \neg(\neg \alpha \vee \neg \beta)$
 - equivalence : $\alpha \equiv \beta \stackrel{def}{=} (\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$.

Propositional Calculus (cont.)

The remaining symbols are defines as:

- the truth constant $\perp$: $\perp \stackrel{def}{=} \neg \top$
- logical connectives:
 - disjunction : $\alpha \vee \beta \stackrel{def}{=} \neg \alpha \rightarrow \beta$
 - conjunction : $\alpha \wedge \beta \stackrel{def}{=} \neg(\neg \alpha \vee \neg \beta)$
 - equivalence : $\alpha \equiv \beta \stackrel{def}{=} (\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$.

Propositional Calculus (cont.)

The remaining symbols are defines as:

- the truth constant $\perp$: $\perp \stackrel{def}{=} \neg \top$
- logical connectives:
 - disjunction : $\alpha \vee \beta \stackrel{def}{=} \neg \alpha \rightarrow \beta$
 - conjunction : $\alpha \wedge \beta \stackrel{def}{=} \neg(\neg \alpha \vee \neg \beta)$
 - equivalence : $\alpha \equiv \beta \stackrel{def}{=} (\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$.

Propositional Calculus (cont.)

The remaining symbols are defines as:

- the truth constant $\perp$: $\perp \stackrel{def}{=} \neg \top$
- logical connectives:
 - disjunction : $\alpha \vee \beta \stackrel{def}{=} \neg \alpha \rightarrow \beta$
 - conjunction : $\alpha \wedge \beta \stackrel{def}{=} \neg(\neg \alpha \vee \neg \beta)$
 - equivalence : $\alpha \equiv \beta \stackrel{def}{=} (\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$.

Semantics of *PC*

Let $\mathcal{L}$ be a language of propositional logic. An *interpretation* of $\mathcal{L}$ is a mapping

$$m : Var \rightarrow \{0, 1\}$$

The mapping m is easily extended for the set $\mathcal{L}$ of all formulas.

Semantics of *PC*

Let $\mathcal{L}$ be a language of propositional logic. An *interpretation* of $\mathcal{L}$ is a mapping

$$m : Var \rightarrow \{0, 1\}$$

The mapping m is easily extended for the set $\mathcal{L}$ of all formulas.

A formula $\alpha \in \mathcal{L}$ is *true in m* (m is a *model* of α), in symbols $m \models \alpha$, iff $m(\alpha) = 1$.

Semantics of PC

Let $\mathcal{L}$ be a language of propositional logic. An **interpretation** of $\mathcal{L}$ is a mapping

$$m : Var \rightarrow \{0, 1\}$$

The mapping m is easily extended for the set $\mathcal{L}$ of all formulas.

A formula $\alpha \in \mathcal{L}$ is **true in m** (m is a **model** of α), in symbols $m \models \alpha$, iff $m(\alpha) = 1$.

Let $\alpha \in \mathcal{L}$ be a formula. We say that α is

- **satisfiable** iff it has a model
- **tautology**, written $\models \alpha$, iff every interpretation of $\mathcal{L}$ is a model of α
- **unsatisfiable** if it has no model.

Most famous tautologies

$$\models \alpha \vee \neg \alpha$$

$$\models \neg(\alpha \wedge \beta) \equiv \neg \alpha \vee \neg \beta$$

$$\models \neg(\alpha \vee \beta) \equiv \neg \alpha \wedge \neg \beta$$

$$\models \neg \neg \alpha \equiv \alpha$$

$$\models \alpha \vee (\beta \wedge \gamma) \equiv (\alpha \vee \beta) \wedge (\alpha \vee \gamma)$$

$$\models \alpha \wedge (\beta \vee \gamma) \equiv (\alpha \wedge \beta) \vee (\alpha \wedge \gamma)$$

$$\models \alpha \wedge \beta \equiv \beta \wedge \alpha$$

$$\models \alpha \vee \beta \equiv \beta \vee \alpha$$

$$\models \alpha \rightarrow \beta \equiv \neg \beta \rightarrow \neg \alpha$$

Excluded Middle Law

De Morgan Law

De Morgan Law

Double Negation Law

Distributive Law

Distributive Law

Commutative Law

Commutative Law

Contraposition Law

Validity Problem

The validity problem is the task to determine whether or not a given formula is a tautology.

Validity Problem

The validity problem is the task to determine whether or not a given formula is a tautology.

In virtue of the truth-table method we have the following:

Theorem 5.1 The validity problem for classical propositional calculus is **decidable**.

Theories

- Let $\mathcal{L}$ be a language of (classical) propositional logic. Any subset $T \subseteq \mathcal{L}$ is called a *theory*.

Theories

- Let $\mathcal{L}$ be a language of (classical) propositional logic. Any subset $T \subseteq \mathcal{L}$ is called a *theory*.
- For a set $T \subseteq \mathcal{L}$ of formulas and the set $\mathfrak{M}$ of interpretations of $\mathcal{L}$, $\mathfrak{M} \models T$, means that every formula $\alpha \in T$ is true in every interpretation $m \in \mathfrak{M}$.

Theories

- Let $\mathcal{L}$ be a language of (classical) propositional logic. Any subset $T \subseteq \mathcal{L}$ is called a **theory**.
- For a set $T \subseteq \mathcal{L}$ of formulas and the set $\mathfrak{M}$ of interpretations of $\mathcal{L}$, $\mathfrak{M} \models T$, means that every formula $\alpha \in T$ is true in every interpretation $m \in \mathfrak{M}$.
- Two theories $T_1, T_2 \subseteq \mathcal{L}$ are called **equivalent**, written $T_1 \Leftrightarrow T_2$, iff $Mod(T_1) = Mod(T_2)$, where $Mod(T)$ is a set of all models of T .

Reasoning rules

A *reasoning rule*

$$r = \frac{\alpha_1, \dots, \alpha_n}{\gamma}$$

is a partial mapping $r : \mathcal{L}^n \rightarrow \mathcal{L}$. For $\alpha_1, \dots, \alpha_n$ from the domain of r , $\alpha_1, \dots, \alpha_n$ are *premises* of r and $\gamma = r(\alpha_1, \dots, \alpha_n)$ is the *consequence* of r .

Reasoning rules

A *reasoning rule*

$$r = \frac{\alpha_1, \dots, \alpha_n}{\gamma}$$

is a partial mapping $r : \mathcal{L}^n \rightarrow \mathcal{L}$. For $\alpha_1, \dots, \alpha_n$ from the domain of r , $\alpha_1, \dots, \alpha_n$ are *premises* of r and $\gamma = r(\alpha_1, \dots, \alpha_n)$ is the *consequence* of r .

A reasoning rule $\frac{\alpha_1, \dots, \alpha_n}{\gamma}$ is called *sound* iff $\{\alpha_1, \dots, \alpha_n\} \models \gamma$.

Deduction systems

A *deduction system* (*axiomatization*) is a triple $DS = (\mathcal{L}, \mathcal{A}, \mathcal{R})$, where

- $\mathcal{L}$ is a language of propositional logic,
- $\mathcal{A} \subseteq \mathcal{L}$ is the set of *logical axioms*, and
- $\mathcal{R}$ is the set of reasoning rules.

Deduction systems

A **deduction system** (**axiomatization**) is a triple $DS = (\mathcal{L}, \mathcal{A}, \mathcal{R})$, where

- $\mathcal{L}$ is a language of propositional logic,
- $\mathcal{A} \subseteq \mathcal{L}$ is the set of **logical axioms**, and
- $\mathcal{R}$ is the set of reasoning rules.

DS is called **sound** iff

- each formula $\alpha \in \mathcal{A}$ is a tautology
- each reasoning rule $r \in \mathcal{R}$ is sound.

Derivability

Let $DS = (\mathcal{L}, \mathcal{A}, \mathcal{R})$ be an axiomatization, $T \subseteq \mathcal{L}$, and let $\alpha \in \mathcal{L}$.

- A **formal proof** of α in DS from T is a sequence $(\alpha_0, \dots, \alpha_k)$ of formulas such that
 - $\alpha_0 \in \mathcal{A} \cup T$
 - $\alpha_n = \alpha$
 - for every $i = 1, \dots, k$, either $\alpha_i \in \mathcal{A} \cup T$ or α_i is a direct consequence of $\alpha_0, \dots, \alpha_{i-1}$ wrt some reasoning rule $r \in \mathcal{R}$.

Derivability

Let $DS = (\mathcal{L}, \mathcal{A}, \mathcal{R})$ be an axiomatization, $T \subseteq \mathcal{L}$, and let $\alpha \in \mathcal{L}$.

- A **formal proof** of α in DS from T is a sequence $(\alpha_0, \dots, \alpha_k)$ of formulas such that
 - $\alpha_0 \in \mathcal{A} \cup T$
 - $\alpha_n = \alpha$
 - for every $i = 1, \dots, k$, either $\alpha_i \in \mathcal{A} \cup T$ or α_i is a direct consequence of $\alpha_0, \dots, \alpha_{i-1}$ wrt some reasoning rule $r \in \mathcal{R}$.
- α is called **derivable** from T wrt DS , written $T \vdash_{DS} \alpha$, iff there exists a formal proof of α from T in DS .

Derivability

Let $DS = (\mathcal{L}, \mathcal{A}, \mathcal{R})$ be an axiomatization, $T \subseteq \mathcal{L}$, and let $\alpha \in \mathcal{L}$.

- A **formal proof** of α in DS from T is a sequence $(\alpha_0, \dots, \alpha_k)$ of formulas such that
 - $\alpha_0 \in \mathcal{A} \cup T$
 - $\alpha_n = \alpha$
 - for every $i = 1, \dots, k$, either $\alpha_i \in \mathcal{A} \cup T$ or α_i is a direct consequence of $\alpha_0, \dots, \alpha_{i-1}$ wrt some reasoning rule $r \in \mathcal{R}$.
- α is called **derivable** from T wrt DS , written $T \vdash_{DS} \alpha$, iff there exists a formal proof of α from T in DS .
- Derivability operator Th : for any $T \subseteq \mathcal{L}$, $Th(T) = \{\alpha : T \vdash_{DS} \alpha\}$.

Derivability

Let $DS = (\mathcal{L}, \mathcal{A}, \mathcal{R})$ be an axiomatization, $T \subseteq \mathcal{L}$, and let $\alpha \in \mathcal{L}$.

- A **formal proof** of α in DS from T is a sequence $(\alpha_0, \dots, \alpha_k)$ of formulas such that
 - $\alpha_0 \in \mathcal{A} \cup T$
 - $\alpha_n = \alpha$
 - for every $i = 1, \dots, k$, either $\alpha_i \in \mathcal{A} \cup T$ or α_i is a direct consequence of $\alpha_0, \dots, \alpha_{i-1}$ wrt some reasoning rule $r \in \mathcal{R}$.
- α is called **derivable** from T wrt DS , written $T \vdash_{DS} \alpha$, iff there exists a formal proof of α from T in DS .
- Derivability operator Th : for any $T \subseteq \mathcal{L}$, $Th(T) = \{\alpha : T \vdash_{DS} \alpha\}$.
- T is **consistent** iff $T \not\vdash \alpha$ for some formula α .

Derivability

Let $DS = (\mathcal{L}, \mathcal{A}, \mathcal{R})$ be an axiomatization, $T \subseteq \mathcal{L}$, and let $\alpha \in \mathcal{L}$.

- A **formal proof** of α in DS from T is a sequence $(\alpha_0, \dots, \alpha_k)$ of formulas such that
 - $\alpha_0 \in \mathcal{A} \cup T$
 - $\alpha_n = \alpha$
 - for every $i = 1, \dots, k$, either $\alpha_i \in \mathcal{A} \cup T$ or α_i is a direct consequence of $\alpha_0, \dots, \alpha_{i-1}$ wrt some reasoning rule $r \in \mathcal{R}$.
- α is called **derivable** from T wrt DS , written $T \vdash_{DS} \alpha$, iff there exists a formal proof of α from T in DS .
- Derivability operator Th : for any $T \subseteq \mathcal{L}$, $Th(T) = \{\alpha : T \vdash_{DS} \alpha\}$.
- T is **consistent** iff $T \not\vdash \alpha$ for some formula α .

Derivability

Let $DS = (\mathcal{L}, \mathcal{A}, \mathcal{R})$ be an axiomatization, $T \subseteq \mathcal{L}$, and let $\alpha \in \mathcal{L}$.

- A **formal proof** of α in DS from T is a sequence $(\alpha_0, \dots, \alpha_k)$ of formulas such that
 - $\alpha_0 \in \mathcal{A} \cup T$
 - $\alpha_n = \alpha$
 - for every $i = 1, \dots, k$, either $\alpha_i \in \mathcal{A} \cup T$ or α_i is a direct consequence of $\alpha_0, \dots, \alpha_{i-1}$ wrt some reasoning rule $r \in \mathcal{R}$.
- α is called **derivable** from T wrt DS , written $T \vdash_{DS} \alpha$, iff there exists a formal proof of α from T in DS .
- Derivability operator Th : for any $T \subseteq \mathcal{L}$, $Th(T) = \{\alpha : T \vdash_{DS} \alpha\}$.
- T is **consistent** iff $T \not\vdash \alpha$ for some formula α .

Derivability

Let $DS = (\mathcal{L}, \mathcal{A}, \mathcal{R})$ be an axiomatization, $T \subseteq \mathcal{L}$, and let $\alpha \in \mathcal{L}$.

- A **formal proof** of α in DS from T is a sequence $(\alpha_0, \dots, \alpha_k)$ of formulas such that
 - $\alpha_0 \in \mathcal{A} \cup T$
 - $\alpha_n = \alpha$
 - for every $i = 1, \dots, k$, either $\alpha_i \in \mathcal{A} \cup T$ or α_i is a direct consequence of $\alpha_0, \dots, \alpha_{i-1}$ wrt some reasoning rule $r \in \mathcal{R}$.
- α is called **derivable** from T wrt DS , written $T \vdash_{DS} \alpha$, iff there exists a formal proof of α from T in DS .
- Derivability operator Th : for any $T \subseteq \mathcal{L}$, $Th(T) = \{\alpha : T \vdash_{DS} \alpha\}$.
- T is **consistent** iff $T \not\vdash \alpha$ for some formula α .

Soundness and Completeness

Let $DS = (\mathcal{L}, \mathcal{A}, \mathcal{R})$ be an axiomatization.

- DS is **sound** iff for every theory $T \subseteq \mathcal{L}$ and for every $\alpha \in \mathcal{L}$,
$$T \vdash_{DS} \alpha \implies T \models \alpha$$

Soundness and Completeness

Let $DS = (\mathcal{L}, \mathcal{A}, \mathcal{R})$ be an axiomatization.

- DS is **sound** iff for every theory $T \subseteq \mathcal{L}$ and for every $\alpha \in \mathcal{L}$,
$$T \vdash_{DS} \alpha \implies T \models \alpha$$
- DS is called **complete** iff for every theory $T \subseteq \mathcal{L}$ and for every formula $\alpha \in \mathcal{L}$, $T \models \alpha \implies T \vdash_{DS} \alpha$.

Axiomatization (cont.)

The most popular (sound and complete) axiomatization of propositional logic:

• logical axioms:

- $\top$
- $\alpha \rightarrow (\beta \rightarrow \alpha)$
- $(\alpha \rightarrow (\beta \rightarrow \gamma)) \rightarrow ((\alpha \rightarrow \beta) \rightarrow (\alpha \rightarrow \gamma))$
- $(\alpha \rightarrow \beta) \rightarrow ((\alpha \rightarrow \neg\beta) \rightarrow \neg\alpha)$

Axiomatization (cont.)

The most popular (sound and complete) axiomatization of propositional logic:

• logical axioms:

• $\top$

• $\alpha \rightarrow (\beta \rightarrow \alpha)$

• $(\alpha \rightarrow (\beta \rightarrow \gamma)) \rightarrow ((\alpha \rightarrow \beta) \rightarrow (\alpha \rightarrow \gamma))$

• $(\alpha \rightarrow \beta) \rightarrow ((\alpha \rightarrow \neg\beta) \rightarrow \neg\alpha)$

• Reasoning rule: **Modus Ponens** $\frac{\alpha, \alpha \rightarrow \beta}{\beta}$.

Axiomatization (cont.)

The most popular (sound and complete) axiomatization of propositional logic:

• logical axioms:

• $\top$

• $\alpha \rightarrow (\beta \rightarrow \alpha)$

• $(\alpha \rightarrow (\beta \rightarrow \gamma)) \rightarrow ((\alpha \rightarrow \beta) \rightarrow (\alpha \rightarrow \gamma))$

• $(\alpha \rightarrow \beta) \rightarrow ((\alpha \rightarrow \neg\beta) \rightarrow \neg\alpha)$

• Reasoning rule: **Modus Ponens** $\frac{\alpha, \alpha \rightarrow \beta}{\beta}$.

Axiomatization (cont.)

The most popular (sound and complete) axiomatization of propositional logic:

• logical axioms:

• $\top$

• $\alpha \rightarrow (\beta \rightarrow \alpha)$

• $(\alpha \rightarrow (\beta \rightarrow \gamma)) \rightarrow ((\alpha \rightarrow \beta) \rightarrow (\alpha \rightarrow \gamma))$

• $(\alpha \rightarrow \beta) \rightarrow ((\alpha \rightarrow \neg\beta) \rightarrow \neg\alpha)$

• Reasoning rule: **Modus Ponens** $\frac{\alpha, \alpha \rightarrow \beta}{\beta}$.

Axiomatization (cont.)

The most popular (sound and complete) axiomatization of propositional logic:

• logical axioms:

• $\top$

• $\alpha \rightarrow (\beta \rightarrow \alpha)$

• $(\alpha \rightarrow (\beta \rightarrow \gamma)) \rightarrow ((\alpha \rightarrow \beta) \rightarrow (\alpha \rightarrow \gamma))$

• $(\alpha \rightarrow \beta) \rightarrow ((\alpha \rightarrow \neg\beta) \rightarrow \neg\alpha)$

• Reasoning rule: **Modus Ponens** $\frac{\alpha, \alpha \rightarrow \beta}{\beta}$.

Axiomatization (cont.)

The most popular (sound and complete) axiomatization of propositional logic:

• logical axioms:

• $\top$

• $\alpha \rightarrow (\beta \rightarrow \alpha)$

• $(\alpha \rightarrow (\beta \rightarrow \gamma)) \rightarrow ((\alpha \rightarrow \beta) \rightarrow (\alpha \rightarrow \gamma))$

• $(\alpha \rightarrow \beta) \rightarrow ((\alpha \rightarrow \neg\beta) \rightarrow \neg\alpha)$

• Reasoning rule: **Modus Ponens** $\frac{\alpha, \alpha \rightarrow \beta}{\beta}$.

Thank you for your attention!

Any questions are welcome.